SRE Lab

A hands-on DevOps and SRE troubleshooting lab with five full-stack applications on Amazon EKS. The applications use Amazon RDS for PostgreSQL, public traffic enters through Route 53 and a shared Application Load Balancer, images live in ECR, and Datadog provides metrics, events, logs, traces, dashboards, and monitors.

The training loop is:

```
Datadog symptom -> scope -> Kubernetes or AWS evidence -> root cause -> fix -> Datadog recovery
```

> **Joining a cluster your instructor already launched?** Do **not** run anything below. You don't need AWS, Terraform, or Docker. Go straight to Shared cluster access â†’ For participants.

Launch the lab

Admin only. Complete these steps in order. The deployment creates billable AWS resources, so review the prerequisites and keep the teardown command available before you begin.

1. Create the Datadog keys before launching

The lab requires both keys at launch:

- `DATADOG_API_KEY` lets the Datadog Agent send metrics, events, logs, and traces.
- `DATADOG_APP_KEY` lets the setup script import the included dashboards and monitors and provision Real User Monitoring (RUM).

In Datadog, create or obtain an API key and an application key for your account. Store them securely and never add them to `terraform.tfvars`, a source file, or Git. Confirm the correct Datadog site for your account, such as `datadoghq.com` or `datadoghq.eu`.

2. Check the prerequisites

- Bash 4.4 or newer
- Terraform, AWS CLI, kubectl, Docker, Helm, jq, `envsubst`, `curl`, and `nslookup`
- An AWS account with credentials configured
- An existing Route 53 public hosted zone with registrar delegation already configured
- A Datadog account, API key, and application key
- Budget for real EKS, EC2, NAT Gateway, ALB, Route 53, and RDS resources

The fixed resource names support one lab deployment per AWS account. Run the commands from Bash on Linux, WSL, or macOS. macOS users need a newer Bash than the system Bash 3.2.

Verify the most important connections before launching:

```
aws sts get-caller-identity
docker version
terraform version
kubectl version --client
helm version
```

3. Configure the domain

```
cp terraform/terraform.tfvars.example terraform/terraform.tfvars
```

Open `terraform/terraform.tfvars` and set `dns_zone_name` to the Route 53 public hosted zone that will host the five application records.

4. Launch the complete environment

Replace the placeholder values and run the command from the repository root. Do not save real keys in the command history of a shared computer.

```
DATADOG_API_KEY=<api-key> \
DATADOG_APP_KEY=<app-key> \
DATADOG_SITE=datadoghq.com \
  ./scripts/setup.sh
```

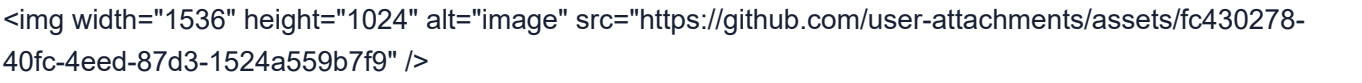
`setup.sh` validates prerequisites, provisions the Terraform resources, builds and pushes ten images, initializes the application databases, deploys the Kubernetes workloads, configures ALB and Route 53, installs metrics-server, and installs Datadog. When both Datadog keys are supplied, it also imports the included dashboards and monitors and provisions RUM. The script prints each application URL when it finishes.

5. Verify the deployment

```
aws eks update-kubeconfig --name dev-cluster --region us-east-1
kubectl get nodes
kubectl get deployments -A
kubectl get pods -A
```

All application Deployments should have their desired Ready replicas. Open the five URLs printed by `setup.sh`, then confirm that data appears in the Datadog application dashboards and **SRE Lab Scenario Signals** before triggering an incident.

Architecture



Five applications share the EKS cluster: `ecommerce`, `banking`, `food-delivery`, `student-portal`, and `support-tickets`. Each has a React/nginx frontend and Node.js/Express backend. See Architecture for the full design and training tradeoffs.

Repository layout

```
apps/                Application source and Kubernetes manifests
datadog/             Helm values, dashboards, and monitors
docs/                Student, scenario, runbook, and PDF material
ingress/             One hostname-based Ingress per application
namespaces/          Namespaces, quotas, and limit ranges
rbac/                Shared-cluster access model: roles, and the per-user sandbox template
scripts/             Setup, teardown, user management, PDF generation, and chaos triggers
terraform/           VPC, EKS, ECR, RDS, Route 53, IAM, and ALB controller
```

Access applications

The configured domain is saved in `.lab-domain`.

```
open https://ecommerce.$(cat .lab-domain)
open https://banking.$(cat .lab-domain)
open https://food-delivery.$(cat .lab-domain)
open https://student-portal.$(cat .lab-domain)
open https://support-tickets.$(cat .lab-domain)
```

Shared cluster access

Many people can use one deployed cluster without needing an AWS account. The admin gives them a kubeconfig file. There are two ways to hand these out:

	Shared class file (recommended for large groups)	**Personal file per person** (small groups)
Created with	`./scripts/users.sh add class --readonly`	`./scripts/users.sh add alice bob ...`
Files to distribute	**One**, the same for everyone	One per person
Read everything (pods, services, Deployments, Ingresses, nodes, ConfigMaps, events, logs, `top`)	Yes	Yes
Read Secrets (Datadog keys, database passwords)	**No**	**No**
Change anything on the shared apps	**No**	**No**
Personal namespace to practice in	No	Yes (`sandbox-<name>`)
Revoke one person	No (rotate the whole file)	Yes

Both are read-only on the five apps and on `kube-system` / `datadog`, and neither can create Ingresses, LoadBalancer/NodePort Services, volumes or RBAC, so nobody can hijack the shared ALB or create billable AWS resources. Personal sandboxes each reserve capacity, so **don't use them for more than about 10 people on the default three-node cluster.**

Read-only access means participants cannot run the incident scenarios against the shared apps. The admin triggers those, and everyone observes and diagnoses.

For participants

Your admin sends you a kubeconfig file (`class.kubeconfig`, or `<your-name>.kubeconfig`). **It contains a credential.** Keep it out of Git and don't repost it publicly.

Save it somewhere outside any repository, then point `kubectl` at it:

```
# Bash / WSL / macOS
export KUBECONFIG=~/.kube/sre-lab.kubeconfig
```

```
# PowerShell
$env:KUBECONFIG = "$HOME\.kube\sre-lab.kubeconfig"
```

```
kubectl get nodes
kubectl get pods -A                     # all pods, all namespaces
kubectl get services -A
kubectl get deployments,ingress,hpa -A
kubectl get all -n banking
kubectl get events -n banking
kubectl describe pod -n ecommerce -l app=ecommerce-backend
kubectl logs -n banking deploy/banking-backend
kubectl top pods -A
```

Everything above is read-only. A `Forbidden` error when you try to create, edit or delete something on the shared apps, or to read Secrets, is expected: that is the access model working.

If you got a personal `<your-name>.kubeconfig`, your default namespace is `sandbox-<your-name>`, so you can also practice there:

```
kubectl run hello --image=nginx
kubectl get pods
kubectl delete pod hello
```

Sandboxes are capped at about five small pods and cannot use LoadBalancer Services or volumes.

The file has this shape. You never have to write it by hand, because `users.sh` generates it:

```
apiVersion: v1
kind: Config
clusters:
  - name: dev-cluster
    cluster:
      server: https://<cluster-endpoint>            # public
      certificate-authority-data: <base64-ca>       # public
users:
  - name: <your-name>
    user:
      token: "<your-personal-token>"                # SECRET: this is your login
contexts:
  - name: dev-cluster
    context: { cluster: dev-cluster, user: <your-name>, namespace: sandbox-<your-name> }
current-context: dev-cluster
```

This README does not include a working kubeconfig on purpose. The endpoint changes every time the cluster is rebuilt, and the token is a credential: anything committed to a repository or README is readable by everyone who can see it.

For the cluster admin

Run these from the machine that ran `setup.sh`, using the identity that created the cluster:

```
aws eks update-kubeconfig --name dev-cluster --region us-east-1

./scripts/users.sh add class --readonly  # ONE shared read-only file ->
kubeconfigs/class.kubeconfig (use this for 50+ people)
./scripts/users.sh add alice bob carol   # or: one file per person, each with a personal sandbox
./scripts/users.sh list                  # identities, sandboxes, quota usage
./scripts/users.sh remove class          # revoke instantly (--keep-sandbox keeps a personal
sandbox)
```

The script shows which cluster it is about to change and asks before it does anything, because a stale `kubectl` context is an easy way to apply changes to the wrong cluster. Add `-y` to skip the prompt.

- **Distribute through your class channel** (email, LMS, chat), never through Git or a public README. `kubeconfigs/` is gitignored. The shared file grants only read access without Secrets, but anyone holding it can query the public API endpoint, so don't post it publicly. For personal files, send each only to its owner. Re-running `add` re-issues the same file and does not rotate the token. To rotate, run `remove` then `add`, and hand out the new file.
- **Tokens do not expire.** They are revoked by `remove`. Use that when someone leaves.
- **Capacity:** each sandbox is capped at 250m CPU and 320Mi memory in requests. Ten people using their full quota need about 2.5 CPU and 3Gi on top of the apps, and the default is three `t3.medium` nodes. If pods sit `Pending`, raise `node_group_desired_size` in `terraform/terraform.tfvars` and run `terraform apply`. Quotas are in rbac/user-sandbox.yaml.tpl.
- **Datadog data goes to your Datadog account only.** Participants can see raw cluster state with `kubectl`, but they need to be invited to your Datadog organization to use the dashboards and monitors.

Known limits of this model:

- The EKS API endpoint is reachable from the public internet, as configured in terraform/eks.tf. A token is the only barrier, so treat leaked files as incidents.
- The app URLs and their `/api/chaos/*` endpoints are public. Read-only Kubernetes access does not stop someone from running `scripts/chaos/inject-latency.sh`, so agree on who triggers chaos.
- Sandbox pods run on the same nodes as the apps. The namespace enforces the Pod Security `baseline` profile and quotas, but there is no NetworkPolicy, so a sandbox pod can still reach in-cluster Services and the node network. This suits trusted classmates and colleagues, not the open internet.

Datadog

datadog/helm-values.yaml enables APM, container logs, Kubernetes events, infrastructure metrics, and the Cluster Agent. Setup imports every JSON file under `datadog/dashboards/` and `datadog/monitors/` when both Datadog keys are supplied.

Establish a healthy baseline in each application dashboard and **SRE Lab Scenario Signals** before starting an exercise.

Five incident scenarios

Scenario	Main DevOps skill
Bad Container Image Deployment	ECR and Kubernetes Deployments
Bad ConfigMap Rollout	ConfigMaps and health probes
OOMKilled and Resource Limits	Kubernetes resource management
ALB and Ingress Routing Failure	ALB and Kubernetes networking
High Application Latency	Datadog APM and workload troubleshooting

Student scenarios begin with symptoms and Datadog. Production-style STAR versions are in DevOps STAR Scenarios.

Guides and PDFs

- Connecting to the shared cluster, also as PDF
- Student Guide
- Scenario test checklist
- Runbooks
- SLO, SLA, and SLI
- Error budgets
- SRE Lab launch guide PDF
- DevOps STAR scenario PDF
- Combined scenario PDF
- Individual scenario PDFs

The README, DevOps STAR document, and Markdown files under `docs/incident-scenarios/` are the PDF sources. Regenerate PDFs with:

```
powershell -ExecutionPolicy Bypass -File scripts/generate-pdfs.ps1
```

Destroy the environment

The lab costs money while running. Teardown deletes Ingress resources first so the controller can release the ALB, uninstalls Datadog and the controller, removes namespaces, and runs Terraform destroy.

```
./scripts/teardown.sh
```

Afterward, verify that the lab has no remaining ALBs, target groups, NAT Gateways, Elastic IPs, ECR repositories, or RDS instances in the AWS console.